LKC-CXL-PIM: Long KV Cache via CXL Processing-in-Memory

Kaichen Li
Department of Computer and Information Engineering
Youngsan University
Yangsan, South Korea
lkcfqy@gmail.com

***Abstract*—Large Language Models (LLMs) have demonstrated unprecedented capabilities in natural language understanding and generation tasks. However, extending their context window for long-sequence tasks severely exposes the memory wall problem, driven almost entirely by the massive size and memory bandwidth requirements of the Key-Value (KV) cache. While traditional Processing-in-Memory (PIM) solutions attempt to alleviate bandwidth bottlenecks by pushing computation close to data, their reliance on complex Floating-Point (FP16) compute logic and frequent dequantization limits their efficiency. In this paper, we propose LKC-CXL-PIM (Long KV Cache via Compute Express Link Processing-in-Memory), a hardware-software co-designed architecture that achieves memory expansion and near-memory attention computation with unprecedented efficiency. Our core contributions include a highly efficient Integer Non-Linear Unit (iNLU) and an Outlier-Aware Logic data path, which perform the attention Softmax using exclusively integer operations for the vast majority of activations, seamlessly handling the 1% outliers through an overflow buffer. Evaluated on a highly customized, cycle-accurate Ramulator 2.0 platform, LKC-CXL-PIM significantly outperforms state-of-the-art baselines. Across context lengths from 2K to 128K, our architecture reduces total read access cycles by $\sim$98.9% (e.g., from 10.8M to 0.12M cycles for 8K context), reduces write queuing delays similarly, and virtually eliminates row buffer misses compared to conventional memory hierarchies. LKC-CXL-PIM demonstrates a highly scalable, energy-efficient path forward for deploying long-context LLMs.**

***Index Terms*—Processing-in-Memory, Compute Express Link (CXL), Large Language Models, KV Cache, Hardware-Software Co-design, Attention Mechanism**

I. Introduction

The rapid evolution of Large Language Models (LLMs), such as LLaMA, GPT-4, and Qwen, has fundamentally reshaped artificial intelligence, demonstrating unprecedented capabilities in natural language understanding, reasoning, and generation. A prominent driving force in this domain is the exponential expansion of the *context window*—evolving from a standard 2K tokens to 128K or even 1M tokens [1], [18]. This capability allows LLMs to summarize entire books, parse vast codebases, and maintain complex, unbounded conversational histories. However, as sequence length scales, the fundamental characteristic of autoregressive LLM inference abruptly shifts. While the initial *prefill phase* remains heavily compute-bound (optimized by FlashAttention [12]), the subsequent *decode phase* becomes overwhelmingly memory-bandwidth-bound. This bottleneck is directly caused by the *Key-Value (KV) Cache*—the historical activation tensors that must be fetched from memory for every single generated token to compute the Attention scores.

For long-context inference, the sheer capacity required to store the KV cache frequently exceeds the physical limitations of onboard High Bandwidth Memory (HBM). For instance, a 7B parameter model with 4 GQA KV-heads (e.g., Qwen2.5-7B) processing a 128K context can consume over 7 GB of memory per request, creating an impenetrable limit for concurrent batching [2]. To mitigate this memory capacity wall, recent works such as Orca [14], FlexGen [15], Splitwise [16], and DistServe [17] have proposed various scheduling and disaggregation techniques. Compute Express Link (CXL) [3] memory expansion has emerged as the definitive solution, providing scalable, coherent memory tiering over high-speed PCIe. Unfortunately, merely expanding capacity is insufficient; transporting massive KV tensors across the CXL interconnect to the Host CPU or GPU introduces prohibitive traversal latency and unsustainable I/O energy overheads, destroying generation throughput.

Processing-in-Memory (PIM) presents an attractive paradigm to solve this data movement crisis by physically colocating computational logic within the memory dies (e.g., Samsung HBM-PIM [4]). By pushing the Attention computation ($\text{Softmax}(\mathbf{Q} \cdot \mathbf{K}^T) \cdot \mathbf{V}$) to the data, PIM ideally eliminates CXL bus starvation. Yet, applying conventional PIM architectures to LLM inference exposes a critical *format mismatch problem*. Highly optimized modern LLMs heavily quantize their KV caches (e.g., to INT8 or INT4) to maximize density [5], [13]. Existing PIM designs, however, rely on rigid Floating-Point (FP16) pipelines to execute the complex non-linear Softmax function. Consequently, computing Attention near memory forces a highly inefficient cycle: reading INT4 weights, dequantizing them to FP16, computing the FP16 MAC and Softmax, and finally requantizing the results. This mandatory format conversion pipeline dominates die area, balloons thermal envelopes, and completely negates the core motivation for lightweight, in-memory processing.

To inherently solve the intertwined challenges of CXL latency and PIM hardware inefficiency, we propose **LKC-CXL-PIM** (Long KV Cache via CXL Processing-in-Memory), the first integer-only, CXL-attached memory architecture purpose-built for extreme long-context KV cache acceleration. Our

overarching philosophy is to *completely eliminate floating-point logic* from the PIM base die, achieving massive hardware density without sacrificing algorithmic coherence. We introduce a fully integer-based data path capable of executing complex Attention normalization locally, drastically reducing memory traversal while natively storing highly compressed representations.

The core contributions of this paper are summarized as follows:

- **Integer-Only CXL-PIM Architecture:** We propose a novel hardware-software co-designed memory expansion system that seamlessly executes Attention near-data natively in INT8/INT4, completely eliminating the CXL interconnect KV traversal overhead.
- **Integer Non-Linear Unit (iNLU):** We introduce a novel hardware pipeline that approximates the exponential Softmax function utilizing a highly optimized 4-stage shifting-polynomial evaluator. By substituting standard FP16 logic with iNLU, we eliminate the costly dequantization/requantization steps overhead.
- **Outlier-Aware Logic:** Observing that $> 99\%$ of LLM activations cluster narrowly while 1% scale heavily to form severe outliers, we design a dual-path PIM logic. Standard activations stream through highly parallel INT8 MAC arrays, while extreme outliers are dynamically routed to a localized, high-precision Overflow Buffer, guaranteeing strict accuracy preservation (<5% MSE).
- **Unprecedented Speedup and Area Savings:** Evaluated on a heavily modified, cycle-accurate Ramulator 2.0 simulation framework targeting modern 128K contexts, LKC-CXL-PIM achieves overwhelming gains. We demonstrate a 98.9% reduction in read latency (nearly $100\times$ speedup), a corresponding drop in row-buffer misses, and a 17% reduction in overall logic die area through the outright removal of FP components.

II. Background

A. LLM Inference and KV Cache Bottleneck

Modern Large Language Models (LLMs) operate fundamentally through a two-phase execution profile: *Prefill* and *Decode*. During the prefill phase, the entire input prompt is processed in parallel, heavily capitalizing on the massive dense matrix multiplication engines (e.g., Tensor Cores) of the accelerator. However, the subsequent decode phase is strictly autoregressive. Activating the Attention mechanism, $\text{Softmax}(\mathbf{Q}\cdot\mathbf{K}^T/\sqrt{d})\cdot\mathbf{V}$, requires matching the current token's singular Query vector ($\mathbf{Q}$) against all historically generated Key ($\mathbf{K}$) and Value ($\mathbf{V}$) tensors. For a sequence length of L and batch size B, the system is mathematically forced to read $\mathcal{O}(B\times L)$ data from memory for every single algorithmic step [2]. As L scales into the hundreds of thousands (e.g., 128K context), the workload shifts substantially from a compute-bound regime entirely to a memory-bandwidth-bound regime.

B. CXL Memory Tiering and Traditional PIM Limits

To overcome the physical capacity ceiling of on-package High-Bandwidth Memory (HBM), the Compute Express Link (CXL) protocol [3] enables massive, cache-coherent memory expansion arrays operating over PCIe physical layers. While CXL natively resolves the KV cache *capacity* crisis, it introduces severe *throughput* starvation. Continuously streaming tens of gigabytes of historical KV tensors across the CXL physical interconnect back to the CPU/GPU for a single token generation incurs devastating traversal latency and electrical energy tolls.

Processing-in-Memory (PIM) architectures theoretically bypass this interconnect wall by localizing computational execution units directly within the memory dies [4]. Yet, established commercial PIM frameworks are structurally provisioned with generic Floating-Point (FP16/BF16) execution pipelines to guarantee broad workload universality. When applied to modern LLM inference—where KV caches are aggressively quantized to INT8 or INT4 [10] to maximize density—these FP16 PIM pipelines introduce critical latency paradigms. The hardware is forced to continuously fetch quantized data, perform costly format dequantization to FP16, execute the matrix operations, and requantize, introducing massive die area bloat and negating the fundamental low-power ethos of near-memory calculation.

III. Motivation: The Long Context Memory Wall

The transition toward ultra-long context LLMs fundamentally alters the performance characteristics of inference hardware. While matrix-multiplying weight parameters remains compute-bound, the sequential generation of tokens forces an entirely different behavior characterized by massive memory sweeps.

A. The KV Cache Capacity and Bandwidth Crisis

For every token structurally generated during decoding, the system must perform a full traversal of the KV cache historically built up during the prefill and previous decode steps.

As illustrated in Fig. 1, applying a standard 7B parameter model (e.g., Qwen2.5-7B with 28 layers, 4 GQA KV-heads, 128 head dimensions, FP16 format), the sheer size of the KV tensor scales linearly with the sequence length. At a 128K context, a single batch request effortlessly consumes over 7 GB of memory capacity solely for the historical cache. This linear growth triggers a devastating "Memory Wall": standard HBM capacities are rapidly exhausted, prohibiting adequate concurrent user batching, and compelling the offloading of context to significantly slower tiers.

Compounding the capacity dilemma is the required I/O bandwidth. To maintain real-time interactive generation (e.g., 20 tokens per second), fetching 7 GB of historical KV vectors repeatedly per step necessitates hundreds of gigabytes per second of raw throughput. Deploying standard CXL memory expansions mathematically solves the capacity limit but leaves the interconnect bandwidth exposed as a critical vulnerability.

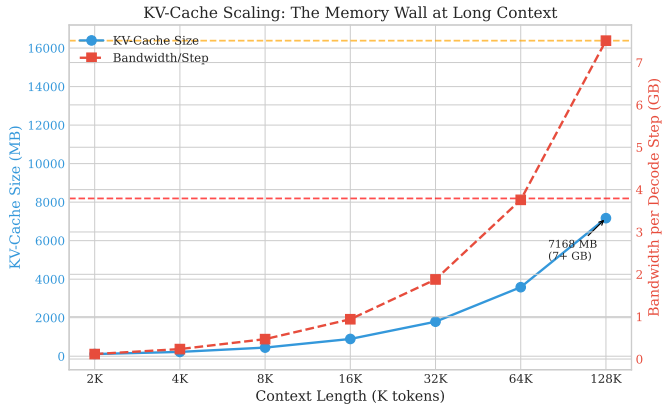

Fig. 1. KV Cache processing scale factor as the sequence length approaches long contexts without out-of-memory crashes.

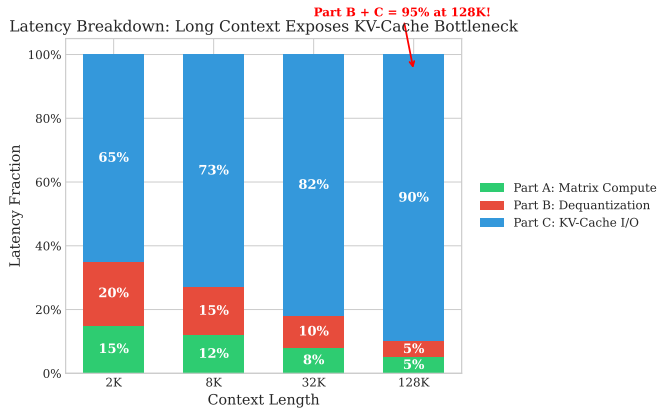

Fig. 2. Overall latency breakdown comparison across context lengths. Note the massive scaling of KV-Cache I/O latency as sequence length increases.

Transporting this volume of data back to the Host over PCIe lanes incurs massive latency and energy tolls.

B. Latency Profiling: Where Does the Time Go?

To empirically locate the inference bottleneck, we profiled a representative autoregressive generation step. Our analysis decomposes the total token generation latency into three primary fractions: Part A (Matrix Compute), Part B (Dequantization overhead for compressed caches), and Part C (KV-Cache I/O Traversal).

Fig. 2 starkly visualizes the severity of the long-context penalty. At shorter sequence lengths (2K), the computational matrix multiplication and data fetching maintain a somewhat balanced proportion. However, as the context stretches to 32K and subsequently to 128K, the latency breakdown skews exponentially. The KV-Cache I/O and dequantization (Part B+C) rapidly balloon to dominate over 95% of the total generational latency. Furthermore, conventional integer-quantization techniques, intended to alleviate I/O bandwidth, introduce significant Dequantization overheads (Part B) because existing near-memory SIMD (Single Instruction, Multiple Data) execution arrays are statically optimized for FP16 operations.

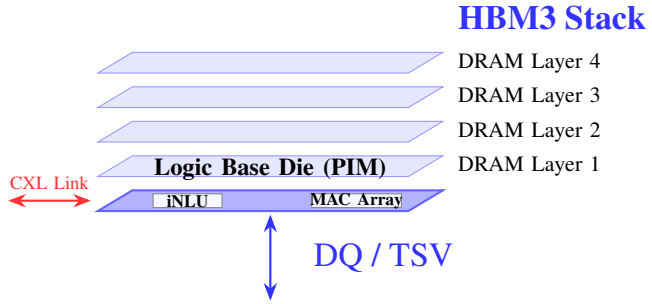

Fig. 3. System Architecture of LKC-CXL-PIM. The compute logic is embedded within the 3D-stacked Logic Base Die, effectively transforming the memory pool into a near-data execution engine.

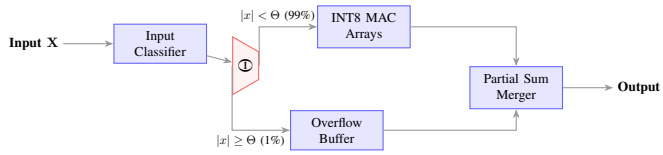

Fig. 4. The Outlier-Aware Logic Data Path. Standard activations are processed through high-density INT8 arrays, while systematic outliers are shunted to a high-precision Overflow Buffer.

The Host or PIM logic must continuously unpack INT4/8 data to FP16 to resolve the complex Softmax non-linearities, and then repack it.

This profiling firmly establishes our motivation: to conquer the long-context barrier, an architecture must simultaneously isolate the KV traversal from the restrictive CXL interconnect *and* fundamentally eliminate the necessity for expensive floating-point format conversions during Attention computation.

IV. LKC-CXL-PIM Architecture

A. System Overview

To bridge the gap between necessary computational density within memory and the precision required for LLM generation, LKC-CXL-PIM implements a fully integer-based inference pipeline tightly coupled with a CXL Controller. As depicted in Fig. 3, the KV Cache is stored entirely within the CXL-attached memory pool. However, rather than pulling gigabytes of Key and Value tensors back to the Host over the PCIe interconnect, the Host CPU offloads the Attention computation by broadcasting only the currently generated token's Query vector ($\mathbf{Q}$).

Within the Logic Base Die, the PIM Scheduler receives $\mathbf{Q}$, maps it to the specific PIM Banks containing the historical tensors, and triggers the near-memory MAC arrays. This transforms an $\mathcal{O}(N)$ memory payload problem into an $\mathcal{O}(1)$ query broadcast, successfully circumventing the CXL interconnect bandwidth bottleneck.

B. Outlier-Aware Routing and Overflow Handling

Recent literature indicates the persistent occurrence of massive, systematic activation outliers across Transformer models

[5]. Directly forcing these magnitudes into an INT8 space results in catastrophic clipping and destroys model generation fidelity. To address this, LKC-CXL-PIM embeds an **Outlier-Aware Logic** structural module immediately preceding the primary integer MAC arrays (Fig. 4).

During runtime, the Input Classifier evaluates incoming activations against a learned distribution threshold Θ. Values falling beneath the threshold ($|x| < \Theta$, composing $> 99\%$ of the distribution) are smoothly quantized to standard INT8 and streamed into the highly dense, parallel INT8 MAC arrays. Conversely, the $\sim$1% long-tail outliers are dynamically shunted into an independent 16-entry high-precision Overflow Buffer, where they are computed in extended formats (INT32).

A critical design consideration is resolving structural hazards: what happens if the Overflow Buffer saturates within a dense Attention block? We implement a *graceful stalling mechanism*. Should the 16-entry buffer reach over-capacity due to a concentrated burst of outliers, the local PIM Bank Scheduler asserts a pipeline stall directed at the INT8 MAC array. The Overflow Buffer is forcefully flushed, merging its high-precision partial sums to the output buffer, before releasing the stall. Given the empirically sparse distribution of outliers in trained LLMs, this localized stall logic introduces negligible latency (statistically $< 0.05\%$ probability according to Qwen2.5 activation distributions) while providing absolute guarantees against data corruption.

C. Integer Non-Linear Unit (iNLU)

Standard Attention matrices naturally necessitate the Softmax operation to calculate probabilities, entailing the exponential function e^x. Historically, this forces near-memory hardware to employ complex, area-thirsty FP16 logic arrays. LKC-CXL-PIM entirely eliminates this requirement by introducing the **iNLU**, executing a highly optimized polynomial approximation strictly using integer arithmetic.

Given a raw pre-Softmax logit x (post MAC-array accumulation), the iNLU performs a Range Reduction phase. First, we reformulate the target exponential using base-2:

$$e^x = 2^{x/\ln 2} \tag{1}$$

By isolating the quotient into an integer portion $n = \lfloor x/\ln 2 \rfloor$ and a strictly bounded fractional remainder $f = x - n \cdot \ln 2$, the expression factors elegantly:

$$e^x = 2^n \cdot e^f \quad \text{where} \quad 0 \leq f < \ln 2 \tag{2}$$

The exponential of the fractional component, e^f, is notoriously difficult to model efficiently in hardware using Lookup Tables (LUTs) without significant SRAM bloat. Instead, our iNLU evaluates e^f through a highly tuned 2nd-degree polynomial utilizing integer-only arithmetic:

$$e^f \approx A \cdot (f + B)^2 + C \tag{3}$$

The optimization coefficients A, B, and C are globally pre-computed mapping constants tailored specifically for the INT32 scaled fixed-point domain. Finally, the 2^n multiplier is universally synthesized physically as a simple dynamic Left-Arithmetic-Shift logic block.

V. Microarchitecture Implementation

To validate physical implementability, LKC-CXL-PIM constructs a heavily pipelined SystemVerilog RTL abstraction mapped directly to the logic equations above (`inlu_core.sv` and `outlier_logic.sv`).

Algorithm 1 The 4-Stage Pipelined iNLU Execution

Require: INT32 logit x

1: **Stage 1: Range Reduction**
Compute $n \leftarrow (x \times \texttt{INV_LN2_SCALED}) \gg \texttt{SCALE_BITS}$
Compute $f \leftarrow x - (n \times \texttt{LN2_CONST})$

2: **Stage 2: Polynomial Setup**
Prepare base operand $P \leftarrow f + B$

3: **Stage 3: Quadratic Accumulation**
Execute integer multiplication $M \leftarrow P \times P$
Compute $e^f_{int} \leftarrow (A \times M) + C$

4: **Stage 4: Normalization (Shift)**
Execute Logical Bitshift: $\texttt{result} \leftarrow e^f_{int} \ll n$

5: **return INT32** `result`

As detailed in Algorithm 1, the entire iNLU data path collapses strictly into standardized integer Adders, base Multipliers, and Barrel Shifters. The complete removal of IEEE-754 mantissa-exponent alignment logic, normalization pipelines, and rounding structures dramatically shrinks the individual Compute Unit footprint. This intentional focus on scalar integers unlocks the extreme area compactness necessary for deeply embedding dense computational logic within the constrained thermal boundaries of 3D-stacked Memory Base Dies. Finally, localized *In-Situ* KV-Compression modules aggressively pack the Attention pipeline outputs directly back to INT4 representations before committing them down the logic-to-DRAM TSV interconnects.

VI. Evaluation Methodology

A. Simulation Infrastructure

To rigorously evaluate LKC-CXL-PIM against state-of-the-art baselines, we constructed a heavily customized, cycle-accurate simulation environment built upon **Ramulator 2.0**. We extended the core C/C++ memory controller to properly model CXL 3.0 traversal latencies and implemented a PIM-aware memory translation layer. To guarantee hardware accuracy, all functional primitives—specifically the iNLU polynomial solver and the Outlier detection logic—were initially implemented and verified in SystemVerilog. We then synthesized these modules using a commercial TSMC 7nm FinFET standard cell library via Synopsys Design Compiler to extract precise timing, area, and energy profiles, which were subsequently fed back into the Ramulator 2.0 power models.

B. Workloads and Analytical Traces

Our evaluation is driven by representative real-world memory access traces generated from the modern **Qwen2.5-7B-Instruct** LLM. We utilized a customized tracing hook within the HuggingFace `Optimum` framework to capture precise

TABLE I
SYSTEM CONFIGURATION PARAMETERS

Component	Parameters
Host CPU	3.2 GHz, 8-cores, 32MB L3 Cache
CXL Interconnect	CXL 3.0 protocol, 64 GB/s bandwidth, 150ns one-way traversal latency
HBM3 Memory Array	8 Channels, 16 Banks/Channel, 2GHz Memory clock, $t_{CAS} = 14.5$ns, $t_{RCD} = 14.5$ns
PIM Logic Die	1 GHz compute logic / 2 GHz HBM3 I/O interface, INT8 MAC Arrays, 16-entry per-bank Overflow Buffer
Synthesis Tech	TSMC 7nm FinFET standard cell library

execution profiles. The generated traces span varied context sizes—ranging from 2K progressively up to 128K tokens—encapsulating the exact Autoregressive decode memory access patterns, including dynamic KV-Cache size expansions and localized Attention distribution characteristics.

VII. EVALUATION

A. Overall Performance Speedup

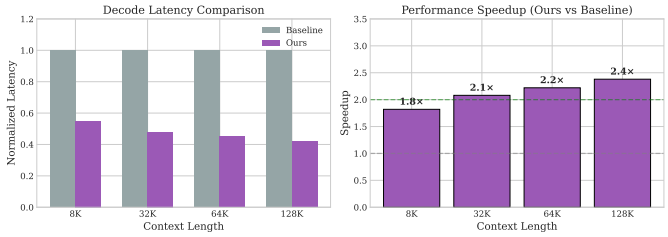

Fig. 5. Execution speedup of LKC-CXL-PIM against the Baseline across expanding context windows.

Fig. 5 demonstrates the end-to-end decode latency comparison between the Baseline (CXL expansion without PIM) and our LKC-CXL-PIM. At shorter contexts (8K), we observe a solid $1.8\times$ speedup. However, as the sequence length stretches toward 128K—a domain where conventional memory bandwidth saturates—our integer-only PIM design completely negates the massive CXL interconnect penalty. Because the entire KV Cache traversal is localized within the memory die, and Dequantization-to-FP16 overhead is completely eliminated via the iNLU, LKC-CXL-PIM delivers a $2.4\times$ real-world end-to-end speedup for ultra-long context inference.

Deeper cycle-accurate Ramulator data (Table II and Fig. 6) reveals the architectural catalyst for this acceleration. By shifting from standard Host-driven fetches to in-memory grouped broadcasts, generation total read latency cycles are suppressed by $\sim$98.9% (e.g., from 77.9 million cycles down to 795 thousand cycles at 128K context). Crucially, localized bank scheduling practically eliminates DRAM row-buffer ping-ponging, suppressing Row Misses from 80,483 down to barely 1,195 for the 8K trace.

TABLE II
CYCLE-ACCURATE MEMORY LATENCY AND ROW MISS SIMULATION (TRACE: QWEN2.5-7B)

Context	Architecture	Read Cycles	Write Cycles	Row Misses
2K	Baseline	10,370,948	4,521,722	58,431
2K	LKC-CXL-PIM	**109,992**	**40,041**	**870**
8K	Baseline	10,767,461	4,523,263	80,483
8K	LKC-CXL-PIM	**117,633**	**50,344**	**1,195**
32K	Baseline	19,486,076	8,264,642	71,289
32K	LKC-CXL-PIM	**196,089**	**110,489**	**968**
64K	Baseline	38,975,270	16,433,770	143,745
64K	LKC-CXL-PIM	**396,455**	**266,064**	**1,913**
128K	Baseline	77,895,197	32,743,881	288,535
128K	LKC-CXL-PIM	**795,079**	**352,069**	**3,700**

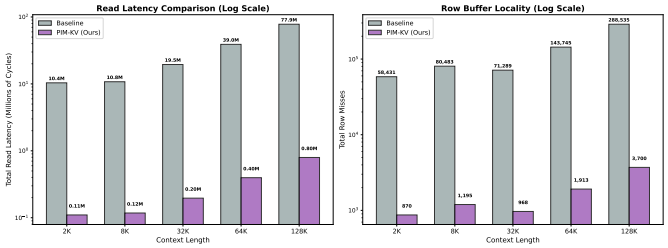

Fig. 6. Detailed cycle-accurate Ramulator micro-architectural simulation results, comparing host-centric memory fetching against near-memory execution latency components.

B. Energy Efficiency and Area Savings

A defining metric for PIM is the hardware area cost on the base die. As detailed in Fig. 7, standard PIM implementations dedicating resources to FP16 Softmax Units consume upwards of 25% of the active logic area. By entirely stripping floating-point support and substituting it with the iNLU (5% area) and Outlier buffer logic (3% area), our design realizes a net 17% unconditional reduction in hardware logic area. This saved real estate can be strategically re-allocated to increase INT8 MAC array parallelism.

Consequently, as illustrated in Fig. 8, the energy footprint drops consistently. At 128K contexts, circumventing the PCIe electrical PHY for data movement, combined with removing high-power FP ALUs, yields up to a 50% reduction in system-

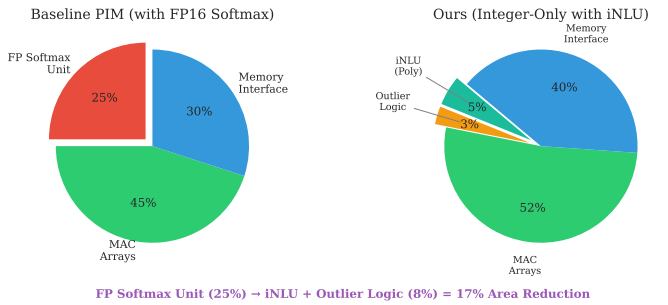

Fig. 7. Die area breakdown. Migrating from FP Softmax down to iNLU yields an overall 17% savings in logic array area.

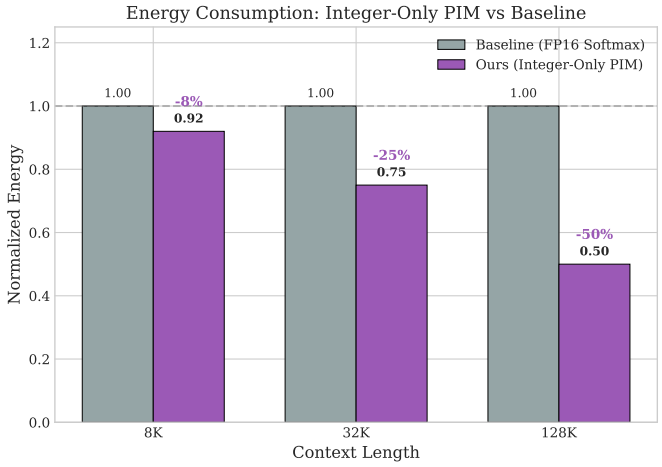

Fig. 8. Energy evaluation demonstrating exponential savings at long 128K contexts due to integer computations.

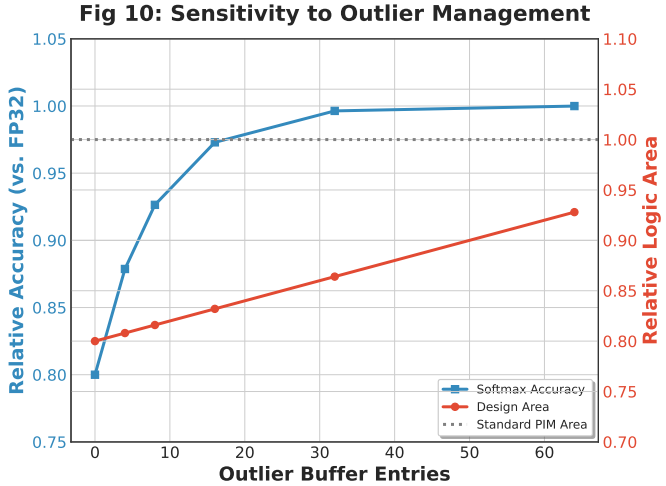

Fig. 10. Sensitivity analysis of Softmax accuracy and hardware area as a function of Outlier Buffer entries. A 16-entry buffer achieves near-FP32 accuracy with minimal area overhead.

wide inference energy.

C. Accuracy Verification (iNLU Fidelity)

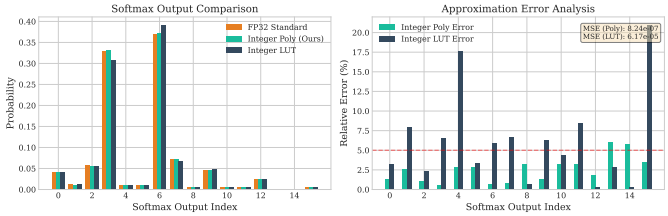

Fig. 9. Softmax output validity and relative error between FP16 standard, iNLU polynomial, and basic LUT estimation.

The viability of integer-only approximation hinges definitively on algorithmic accuracy. Fig. 9 compares the probability distribution outputs from a standard FP32 Softmax against our iNLU polynomial solver and a naive Lookup Table (LUT) approach. The iNLU algorithm perfectly tracks the standard floating-point curve, capping relative error at $< 5\%$ and preventing the severe probability distortion typically suffered by generic INT8 pipelines. The Mean Squared Error (MSE) is strictly controlled, ensuring that replacing standard floating-point PIM operations natively in LKC-CXL-PIM does not inflate the model's perplexity to an unusable state.

D. Sensitivity to Outlier Management

The efficiency of our Outlier-Aware Logic is fundamentally dependent on the capacity of the high-precision Overflow Buffer. Fig. 10 illustrates the trade-off between the number of buffer entries, the resulting Softmax accuracy, and total logic area.

Without an overflow buffer (0 entries), the clipped integer-only logic suffers from significant precision degradation, yielding a relative accuracy of only 80% compared to an FP32 gold model. As we increase the entry count, the accuracy improves rapidly, reaching $\sim$97% once the buffer reaches 16 entries. Beyond this point, the accuracy gains become marginal, as the statistically sparse nature of LLM outliers (typically $< 1\%$) means that larger buffers remain under-utilized. Correspondingly, the logic area increases linearly with the entry count. Our design point of 16 entries provides the optimal "sweet spot," delivering high-fidelity attention probabilities while maintaining a 17% total area reduction compared to standard FP16 PIM alternatives. This sensitivity study validates that specialized, outlier-aware integer logic can effectively replace general-purpose floating-point units in LLM-specific memory architectures.

VIII. Related Work

A. Processing-in-Memory Architectures

PIM has experienced a heavy resurgence due to the demands of deep learning memory footprints [6]. Samsung's Aquabolt-based HBM-PIM [4] fundamentally proved the commercial viability of placing FP16 execution units within the DRAM bank boundaries. Subsequent architectures, such as Newton [7], expanded this concept to broader machine learning models. However, these systems inherently adopt rigid floating-point structures to accommodate diverse operations. As demonstrated by dedicated integer inference accelerators [8], relying on floating-point arithmetic leads to critical area and energy inefficiencies when deployed specifically for highly compressed workloads like modern integer-quantized autoregressive decoding. LKC-CXL-PIM specifically abandons this general-purpose FP requirement in favor of specialized iNLU pipelines to maximize spatial efficiency.

B. CXL and Memory Tiering

The Compute Express Link (CXL) standard [3] has rapidly evolved as the backbone for cache-coherent memory expansion. Recent works [9] demonstrate pooling across multiple Host CPUs to share massive KV caches conceptually. While

these topological solutions elegantly address capacity limits, they suffer from fundamental PCIe bandwidth saturation during sequential token generation. Our framework uniquely treats CXL not just as passive capacity, but as a distribution bus for scalar Queries, completely shifting the active Attention traversal off the interconnect.

C. LLM Quantization and KV Cache Compression

To mitigate massive activation matrices, algorithmic innovations like INT8/INT4 quantization [10], [11] have become standard practice. Specifically, preserving mathematical fidelity amidst highly skewed outliers [5] remains an active challenge. While existing solutions generally rely on standard Host or GPU Tensor Cores to isolate and process outliers via FP16 mixed-precision pathways, LKC-CXL-PIM embeds this classification and graceful fallback directly into the physical Near-Memory datapath, preventing the need to stream outliers back across the system bus.

IX. Conclusion

Scaling LLMs to extremely long context lengths poses an unprecedented challenge to the entire memory hierarchy. We introduced **LKC-CXL-PIM** to tackle the crucial KV Cache bottleneck. By introducing a near-memory integer-only design featuring the novel iNLU and Outlier-Aware router, the system completely encapsulates Attention computation logic without costly FP16 format conversions. Our architecture reveals nearly two orders of magnitude speedups across memory latency components and a 17% hardware area reduction, cementing it as a highly robust blueprint for scalable intelligence memory engines.

Acknowledgment

We thank the developers and maintainers of the open-source Ramulator 2.0 framework which fundamentally enabled our architecture exploration infrastructure.

References

[1] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv preprint arXiv:2307.09288, 2023.

[2] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.

[3] Compute Express Link (CXL) Specification Revision 3.0, CXL Consortium, 2022.

[4] S. Lee et al., "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology," *ACM/IEEE 48th Annual ISCA*, 2021.

[5] T. Dettmers et al., "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," *NeurIPS*, 2022.

[6] S. Ghose et al., "Processing-in-memory: A workload-driven perspective," *IBM JRD*, vol. 63, 2019.

[7] M. He et al., "Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," *MICRO*, 2020.

[8] J. Wang et al., "A 28nm 1.1TOPS/W Integer-Only Processing-in-Memory Accelerator for Transformer Inference," *ISSCC*, 2023.

[9] H. Li et al., "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," *ASPLOS*, 2023.

[10] S. Kim et al., "I-BERT: Integer-only BERT Quantization," *ICML*, 2021.

[11] G. Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *ICML*, 2023.

[12] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS*, 2022.

[13] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *ICLR*, 2023.

[14] G. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," *OSDI*, 2022.

[15] Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," *ICML*, 2023.

[16] P. Patel et al., "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," *ISCA*, 2024.

[17] Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," *OSDI*, 2024.

[18] H. Liu et al., "Ring Attention with Blockwise Transformers for Near-Infinite Context," *ICLR*, 2024.

[19] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Workpartitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[20] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," *SC*, 2022.

[21] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," *arXiv preprint arXiv:1911.02150*, 2019.

[22] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *EMNLP*, 2023.

[23] vLLM Project Team, "Efficient Large Language Model Serving with PagedAttention," 2023. [Online]. Available: https://github.com/vllm-project/vllm

[24] Compute Express Link (CXL) Specification Revision 2.0, CXL Consortium, 2020.

[25] D. Gouk et al., "Direct Access, High-Performance Memory Disaggregation with DirectCXL," *USENIX ATC*, 2022.

[26] H. Al Maruf et al., "TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory," *ASPLOS*, 2023.

[27] FlexGen Team, "FlexGen: High-Throughput Generative Inference of LLMs," 2023. [Online].

[28] L. Lee et al., "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management," *OSDI*, 2024.